

SPÉCIFICATIONS TECHNIQUES

Dossier de conception technique — Logiciel de supervision réseau

Ressource ETRS011 — Module de développement logiciel (32 heures)

Master TRI — Réseaux, Télécommunications, Informatique

Université Savoie Mont Blanc — Campus du Bourget-du-Lac

Rédigé par : Louis ALEXANDRE & Mattis TOINET

Suivi des versions

Ce document est le pendant technique du cahier des charges fonctionnel : il répond au « comment » là où le cahier des charges décrit le « quoi ». Comme lui, c'est un document de travail amené à évoluer ; chaque évolution notable doit être tracée ci-dessous.

Version	Date	Auteur(s)	Description
1.1	24/09/2026	Louis ALEXANDRE & Mattis TOINET	Première version de travail : architecture, choix technologiques et modèle physique.

Sommaire

Suivi des versions.....	2
1. Objet du document.....	4
2. Vue d'ensemble de l'architecture applicative	4
2.1 Principes directeurs	4
2.2 Schéma de composants.....	4
3. Choix technologiques	5
4. Frontend	6
5. Backend et API.....	6
5.1 Organisation du backend.....	6
5.2 Endpoints principaux.....	7
6. Base de données et modèle de données physique.....	7
6.1 Système de gestion de base de données	7
6.2 Schéma physique des tables.....	7
7. Authentification et autorisation	7
8. Flux entre composants	8
8.1 Flux de collecte (polling SNMP)	8
8.2 Flux de consultation (interface web).....	8
9. Architecture réseau et infrastructure de déploiement	8

(Clic droit sur le sommaire ci-dessus puis « Mettre à jour les champs » pour l'actualiser après modification du document.)

1. Objet du document

Le cahier des charges décrit le besoin fonctionnel du projet : le problème adressé, les objectifs, le périmètre et les utilisateurs, sans nommer de technologie. Ce document en est le complément direct : il répond à la question « comment ? » et documente les choix de conception technique qui permettent de réaliser ce périmètre fonctionnel. Il expose l'architecture applicative, les technologies retenues, l'organisation du frontend et du backend, le modèle de données physique, le flux entre composants, et les aspects transverses (sécurité, journalisation, sauvegardes, intégration continue).

Comme le cahier des charges, ce dossier reste un document de travail : les choix qui y sont exposés sont ceux retenus à ce stade du projet, cohérents avec le périmètre et les contraintes qui y sont posées. Ils pourront être révisés au fil de l'avancement, notamment si une contrainte de temps ou une difficulté technique imprévue l'impose.

Ce document ne détaille pas le code lui-même : il se situe entre le cahier des charges fonctionnel et l'implémentation, au niveau où se prennent les décisions structurantes.

2. Vue d'ensemble de l'architecture applicative

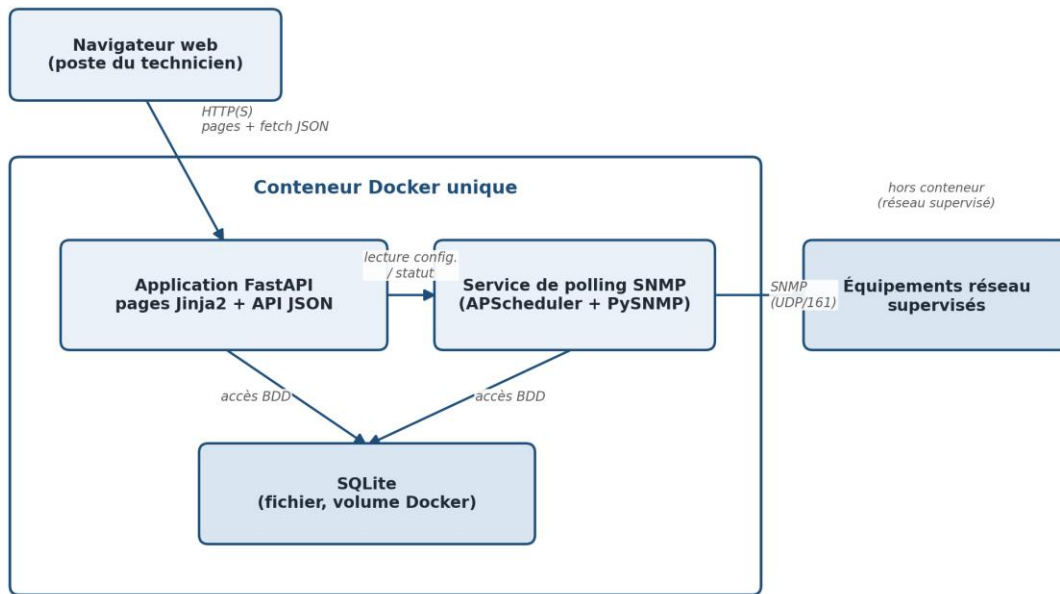
2.1 Principes directeurs

Le projet est développé en binôme durant 32 heures, comprenant la recherche et la documentation. Les choix d'architecture qui suivent découlent avant tout de cette contrainte, avant des considérations de passage à l'échelle qui ne sont pas pertinentes ici :

- **Simplicité avant tout** : une architecture monolithique à trois composants (frontend, API, base de données), plus un service de collecte, plutôt qu'une décomposition en microservices qui multiplierait la complexité opérationnelle sans bénéfice pour un projet de cette taille.
- **Technologies matures et documentées** : chaque brique retenue dispose d'une documentation abondante et d'une communauté large, pour limiter le temps perdu en cas de blocage.
- **Portabilité entre postes personnels** : conformément à l'exclusion posée en section 4.3 du cahier des charges (pas d'hébergement dédié), l'outil doit pouvoir être installé et lancé de façon identique sur les postes Windows, macOS ou Linux des deux membres du binôme.

2.2 Schéma de composants

L'outil tient dans un unique conteneur Docker exécuté sur le poste du technicien, qui embarque trois éléments : l'application FastAPI (pages web et API JSON), le service de polling SNMP, et le fichier de base de données SQLite. Le navigateur du technicien et le parc réseau supervisé sont les deux seuls éléments hors de ce conteneur.



Le détail de chaque composant est développé dans les sections suivantes : le frontend en section 4, l'API et le service de polling en section 5, la base de données en section 6. Les deux flux de données représentés (collecte SNMP d'un côté, consultation web de l'autre) sont détaillés en section 8.

3. Choix technologiques

Le tableau suivant récapitule, pour chaque brique de l'architecture, la technologie retenue, les alternatives envisagées et écartées, et la justification du choix — dans l'esprit des principes directeurs posés en section 2.1.

Composant	Choix retenu	Justification
Langage backend	Python 3	Écosystème SNMP mature en Python (PySNMP), cohérent avec le profil des auteurs (scripting réseau), lisible pour un projet pédagogique.
Framework API	FastAPI	Typage fort natif (Pydantic), documentation OpenAPI générée automatiquement, exécution asynchrone adaptée à l'attente réseau du polling SNMP.
Bibliothèque SNMP	PySNMP	Pure Python, sans dépendance système à compiler : portable entre les postes Windows / macOS / Linux du binôme.
Ordonnancement du polling	APScheduler, intégré au backend	Suffisant pour la volumétrie visée (cf. cahier des charges, section 5.3).
Base de données	SQLite	Zéro administration, fichier unique, largement suffisant pour un parc de démonstration. PostgreSQL ou InfluxDB restent des bascules pertinentes en cas de passage en production (cf. section 15), mais ne se justifient pas pour ce projet.
Frontend	Templates Jinja2 (rendu par FastAPI) + JS minimal	Un SPA et sa chaîne de build (Vite, npm, TypeScript) demandent un temps de mise en place non négligeable pour un gain limité ici ; les pages rendues côté serveur suffisent au périmètre visé et se codent nettement plus vite sur 32 heures.
Graphiques	Chart.js	Aucune installation ni étape de build : un simple <script> suffit ; API simple, largement suffisante pour des courbes temporelles

		(CPU, mémoire, trafic).
Conteneurisation	Docker (conteneur unique)	Un seul service à conteneuriser (SQLite est un fichier, pas un serveur à part) ; portabilité entre postes personnels hétérogènes (cahier des charges, exclusion 4.3) sans complexité d'orchestration inutile.
Intégration continue	GitHub	Intégré à l'hébergement du code, gratuit pour un dépôt étudiant, suffisant pour du lint et des tests automatisés.

4. Frontend

Le frontend n'est pas une application monopage séparée : les pages sont rendues côté serveur par FastAPI lui-même, à l'aide de templates Jinja2, puis renvoyées au navigateur comme du HTML classique. Ce choix évite d'introduire une chaîne de build JavaScript (npm, bundler, TypeScript). Il n'empêche pas pour autant des pages dynamiques, mais déplace le curseur vers le minimum de JavaScript nécessaire plutôt que vers un framework complet.

Chaque écran est donc d'abord une page HTML complète, générée et envoyée par le backend avec les données déjà présentes (pas d'écran vide qui se remplit après coup). Un peu de JavaScript, sans framework, vient ensuite : la bibliothèque Chart.js dessine les graphiques d'historique à partir des données déjà injectées dans la page ou récupérées via un appel fetch vers l'API JSON (section 5.2). Un petit script déclenche par ailleurs un rafraîchissement périodique pour répondre au besoin de rafraîchissement automatique du tableau de bord exprimé en section 4.1 du cahier des charges.

L'application est structurée autour de quatre pages principales, qui correspondent directement aux cas d'utilisation du cahier des charges :

- **Tableau de bord** : vue de synthèse du parc (cas d'utilisation « Superviser le parc »), page rendue à partir des données de GET /api/equipements, rafraîchie périodiquement en JS.
- **Fiche équipement** : détail d'un équipement et historique graphique en Chart.js (cas d'utilisation « Consulter la fiche détaillée » et « Consulter l'historique »), alimentée par GET /api/equipements/{id} et GET /api/equipements/{id}/mesures.
- **Gestion de l'inventaire** : formulaires HTML classiques (soumission via POST) de création, modification et suppression d'équipements (cas d'utilisation « Gérer l'inventaire des équipements »).
- **Gestion des alertes et des seuils** : consultation des alertes actives et configuration des seuils (cas d'utilisation « Gérer les alertes »), à partir de /api/alertes et /api/seuils.

5. Backend et API

5.1 Organisation du backend

Le backend est une application FastAPI unique, qui joue deux rôles à la fois : servir les pages HTML du frontend (section 4, via Jinja2) et exposer une petite API JSON consommée par le JavaScript côté client (Chart.js, rafraîchissement périodique). Ces deux familles de routes partagent la même organisation en trois couches, pour garder la logique de collecte SNMP indépendante de la logique d'exposition HTTP :

- **Couche présentation (routers)** : les routes « pages » rendent des templates Jinja2, les routes /api/* renvoient du JSON ; les deux valident leurs entrées via des schémas Pydantic et délèguent tout traitement à la couche service.
- **Couche service** : logique métier (calcul de statut, application des seuils, logique de corrélation topologique le cas échéant), indépendante du protocole HTTP.

- **Couche accès aux données** : fonctions et requêtes SQL, seule couche qui dialogue avec le fichier SQLite.

Le service de polling SNMP s'exécute dans le même processus backend, ordonnancé par APScheduler selon l'intervalle configuré par équipement, et réutilise la couche service et la couche accès aux données plutôt que de dupliquer la logique métier.

5.2 Endpoints principaux

Le tableau suivant liste les endpoints JSON principaux, ceux que le JavaScript des pages appelle pour les graphiques et le rafraîchissement automatique, et que les formulaires HTML utilisent pour les opérations de création/modification/suppression.

Méthode	Endpoint	Description
GET	/api/equipements	Liste des équipements avec leur statut courant (tableau de bord).
POST	/api/equipements	Création d'un équipement.
GET	/api/equipements/{id}	Détail d'un équipement (fiche équipement).
PUT	/api/equipements/{id}	Modification d'un équipement.
DELETE	/api/equipements/{id}	Suppression d'un équipement (et de son historique associé, cf. cahier des charges 5.6).
GET	/api/equipements/{id}/mesures	Historique des mesures ; paramètres de requête type, depuis, jusqu'à.
GET	/api/equipements/{id}/interfaces	Interfaces réseau déclarées pour l'équipement.
GET	/api/alertes	Liste des alertes ; paramètre de requête statut.
GET	/api/seuils	Liste des seuils configurés.
POST	/api/seuils	Création ou modification d'un seuil.

6. Base de données et modèle de données physique

6.1 Système de gestion de base de données

SQLite est retenu comme moteur de stockage. La base tient dans un fichier unique, monté dans un volume Docker pour persister entre deux lancements du conteneur, sans serveur de base de données à installer ni à administrer. C'est suffisant pour un parc de démonstration de quelques dizaines d'équipements et la volumétrie visée en cahier des charges.

6.2 Schéma physique des tables

À compléter

7. Authentification et autorisation

La gestion multi-utilisateur n'étant pas prise en compte dans la version initial, aucune distinction de droits n'a besoin d'être implémenté. L'outil fonctionnera dans un premier temps avec un acteur implicite unique, sans écran de connexion.

Cela signifie que l'API sera aujourd'hui protégée par aucun mécanisme d'authentification, et ne doit donc être exposée que sur un réseau de confiance, jamais directement sur Internet.

8. Flux entre composants

Deux flux traversent l'architecture ; ils correspondent respectivement au parcours de la donnée décrit fonctionnellement en section 5 du cahier des charges, et à sa restitution.

8.1 Flux de collecte (polling SNMP)

- Le service de polling (APScheduler) déclenche, pour chaque équipement, une interrogation conforme à l'intervalle définie.
- PySNMP envoie les requêtes SNMP GET / GETNEXT vers l'agent de l'équipement, authentifiées par la communauté SNMP enregistrée.
- La réponse est contrôlée (équipement joignable, valeur du type attendu, compteur non aberrant) avant tout traitement, conformément à la gestion des anomalies définie.
- Les métriques dérivées sont calculées (débit par delta de compteurs) puis comparées aux seuils configurés (table seuils_config).
- Le résultat est écrit en base de données.

8.2 Flux de consultation (interface web)

- Le navigateur demande une page (par exemple le tableau de bord) ; FastAPI lit les données nécessaires en base et rend directement le template Jinja2 correspondant, déjà rempli.
- Une fois la page affichée, un script JavaScript minimal interroge à intervalle régulier les endpoints JSON de l'API pour rafraîchir les données affichées sans recharger toute la page.
- Sur la fiche équipement, ce même mécanisme alimente Chart.js, qui restitue l'historique sous forme de courbes à partir des données agrégées renvoyées par l'API pour les périodes longues.
- Les formulaires (inventaire, seuils) soumettent leurs données en POST vers l'API ; FastAPI valide, écrit en base, et renvoie soit la page mise à jour, soit une réponse JSON selon le contexte.

9. Architecture réseau et infrastructure de déploiement

Dans le cadre d'un premier déploiement de type "POC", il n'est pas prévu que l'outil soit hébergé sur une infrastructure dédiée. Le code sera exécuté localement sur les ordinateurs des membres du binôme, dans un unique conteneur Docker. Un volume Docker dédié assure la persistance du fichier SQLite entre deux lancements du conteneur. Un fichier **docker-compose.yml** servira surtout de mémo pour le port exposé, le volume monté et les quelques variables d'environnement.

Deux contraintes réseau découlent de ce mode de déploiement. D'une part, le conteneur doit pouvoir joindre les équipements supervisés en SNMP (UDP/161) : le poste qui l'exécute doit donc être connecté au même réseau que le parc supervisé. D'autre part, l'interface web est exposée sur un port local (par exemple 8080) mappé depuis le conteneur, accessible depuis le poste hôte ; elle n'est pas destinée à être exposée au-delà de ce poste.